Relatório - QuickSort

Comparação de diferentes pivôs e arrays

01 - QuickSort - pivô no meio:

```
public static void quicksortMeio(Geracao g, int[] arr, int esq, int dir) {
    int i = esq, j = dir;
    int pivo = arr[(dir+esq)/2];

    while (i ≤ j) {
        g.comparacoes++;

        while (arr[i] < pivo) {
            g.comparacoes++;
            i++;
        }
        g.comparacoes++;

        while (arr[j] > pivo) {
            g.comparacoes++;
            j--;
        }
        g.comparacoes++;

        g.comparacoes++;
        if (i ≤ j) {
            swap(arr, i, j);
            g.movimentacoes += 3;
            i++;
            j--;
        }
    }
    g.comparacoes++;

    g.comparacoes++;
    if (esq < j)  quicksortMeio(g, arr, esq, j);
    g.comparacoes++;
    if (i < dir)  quicksortMeio(g, arr, i, dir);
}
```

Este QuickSort pega sempre o elemento do meio como pivô.

Array ordenado:

- Tamanho 100: 0.0s / 921 cmp / 189 mov
- Tamanho 1000: 0.001s / 11564 cmp / 1533 mov
- Tamanho 10000: 0.001s / 154959 cmp / 17712 mov

Array aleatório:

- Tamanho 100: 0.0s / 1447 cmp / 540 mov
- Tamanho 1000: 0.001s / 21171 cmp / 7833 mov
- Tamanho 10000: 0.002s / 273809 cmp / 100668 mov

---

02 - QuickSort - pivô no início:

```
public static void quicksortInicio(Geracao g, int[] arr, int esq, int dir) {
    int i = esq, j = dir;
    int pivo = arr[esq];

    while (i <= j) {
        g.comparacoes++;

        while (arr[i] < pivo) {
            g.comparacoes++;
            i++;
        }
        g.comparacoes++;

        while (arr[j] > pivo) {
            g.comparacoes++;
            j--;
        }
        g.comparacoes++;

        g.comparacoes++;
        if (i <= j) {
            swap(arr, i, j);
            g.movimentacoes += 3;
            i++;
            j--;
        }
    }
    g.comparacoes++;

    g.comparacoes++;
    if (esq < j)  quicksortInicio(g, arr, esq, j);
    g.comparacoes++;
    if (i < dir)  quicksortInicio(g, arr, i, dir);
}
```

Este QuickSort pega sempre o primeiro elemento(esquerda) como pivô.

Array ordenado:

- Tamanho 100: 0.0s / 5643 cmp / 297 mov
- Tamanho 1000: 0.002s / 506493 cmp / 2997 mov
- Tamanho 10000: 0.037s / 50064993 cmp / 29997 mov

Array aleatório:

- Tamanho 100: 0.0s / 1788 cmp / 552 mov
- Tamanho 1000: 0.001s / 23871 cmp / 7896 mov
- Tamanho 10000: 0.002s / 302785 cmp / 102057 mov

---

03 - QuickSort - pivô no fim:

```
public static void quicksortFim(Geracao g, int[] arr, int esq, int dir) {
    int i = esq, j = dir;
    int pivo = arr[dir];

    while (i <= j) {
        g.comparacoes++;

        while (arr[i] < pivo) {
            g.comparacoes++;
            i++;
        }
        g.comparacoes++;

        while (arr[j] > pivo) {
            g.comparacoes++;
            j--;
        }
        g.comparacoes++;

        g.comparacoes++;
        if (i <= j) {
            swap(arr, i, j);
            g.movimentacoes += 3;
            i++;
            j--;
        }
    }
    g.comparacoes++;

    g.comparacoes++;
    if (esq < j)  quicksortFim(g, arr, esq, j);
    g.comparacoes++;
    if (i < dir)  quicksortFim(g, arr, i, dir);
}
```

Este QuickSort pega sempre o último elemento(direita) como pivô.

Array ordenado:

- Tamanho 100: 0.0s / 5643 cmp / 297 mov
- Tamanho 1000: 0.006s / 506493 cmp / 2997 mov
- Tamanho 10000: 0.038s / 50064993 cmp / 29997 mov

Array aleatório:

- Tamanho 100: 0.0s / 1752 cmp / 567 mov
- Tamanho 1000: 0.001s / 24554 cmp / 7926 mov
- Tamanho 10000: 0.002s / 318104 cmp / 101430 mov

---

04 - QuickSort - pivô aleatório:

```java
public static void quicksortAleatorio(Geracao g, int[] arr, int esq, int dir) {
    Random r = new Random();
    int i = esq, j = dir;
    int pivo = arr[Math.abs(r.nextInt((dir + 1) - esq) + esq)];

    while (i <= j) {
        g.comparacoes++;

        while (arr[i] < pivo) {
            g.comparacoes++;
            i++;
        }
        g.comparacoes++;

        while (arr[j] > pivo) {
            g.comparacoes++;
            j--;
        }
        g.comparacoes++;

        g.comparacoes++;
        if (i <= j) {
            swap(arr, i, j);
            g.movimentacoes += 3;
            i++;
            j--;
        }
    }
    g.comparacoes++;

    g.comparacoes++;
    if (esq < j)  quicksortAleatorio(g, arr, esq, j);
    g.comparacoes++;
    if (i < dir)  quicksortAleatorio(g, arr, i, dir);
}
```

Este QuickSort pega sempre um elemento aleatório como pivô.

Array ordenado:

- Tamanho 100: 0.0s / 1099 cmp / 207 mov
- Tamanho 1000: 0.002s / 15387 cmp / 2007 mov
- Tamanho 10000: 0.003s / 204730 cmp / 19953 mov

Array aleatório:

- Tamanho 100: 0.0s / 1558 cmp / 588 mov
- Tamanho 1000: 0.0s / 22125 cmp / 7827 mov
- Tamanho 10000: 0.004s / 283712 cmp / 101838 mov

---

05 - QuickSort - pivô mediana:

```java
public static void quicksortMediana(Geracao g, int[] arr, int esq, int dir) {
    int[] mediana = new int[3];
    mediana[0] = esq;
    mediana[1] = (dir + esq) / 2;
    mediana[2] = dir;

    int medianaValor;
    if ((mediana[0] > mediana[1] && mediana[0] < mediana[2]) || (mediana[0] < mediana[1] && mediana[0] > mediana[2])){
        medianaValor = mediana[0];
    }
    else if ((mediana[1] > mediana[0] && mediana[1] < mediana[2]) || (mediana[1] < mediana[0] && mediana[1] > mediana[2])) {
        medianaValor = mediana[1];
    }
    else{
        medianaValor = mediana[2];
    }

    int i = esq, j = dir;
    int pivo = arr[medianaValor];

    while (i <= j) {
        g.comparacoes++;

        while (arr[i] < pivo) {
            g.comparacoes++;
            i++;
        }
        g.comparacoes++;

        while (arr[j] > pivo) {
            g.comparacoes++;
            j--;
        }
        g.comparacoes++;

        g.comparacoes++;
        if (i <= j) {
            swap(arr, i, j);
            g.movimentacoes += 3;
            i++;
            j--;
        }
    }
    g.comparacoes++;

    g.comparacoes++;
    if (esq < j)  quicksortMediana(g, arr, esq, j);
    g.comparacoes++;
    if (i < dir)  quicksortMediana(g, arr, i, dir);
}
```

Este QuickSort pega sempre o elemento correspondente entre a mediana entre o início(esquerda), o final(direita) e o meio, como pivô.

Array ordenado:

- Tamanho 100: 0.0s / 921 cmp / 189 mov
- Tamanho 1000: 0.0s / 11564 cmp / 1533 mov
- Tamanho 10000: 0.001s / 154959 cmp / 17712 mov

Array aleatório:

- Tamanho 100: 0.0s / 1626 cmp / 558 mov
- Tamanho 1000: 0.002s / 22257 cmp / 7650 mov
- Tamanho 10000: 0.003s / 273787 cmp / 100677 mov

Conclusão:

QuickSort com pivô no meio

- Ordenado: bom desempenho, poucas comparações e movimentos.
- Aleatório: desempenho razoável, não é o pior nem o melhor.
- Observação: é estável para arrays parcialmente ordenados.

QuickSort com pivô no início

- Ordenado: pior caso clássico do QuickSort: número de comparações explode ($O(n^2)$).
- Aleatório: desempenho aceitável.
- Observação: usar o primeiro elemento como pivô é arriscado em arrays já ordenados.

QuickSort com pivô no fim

- Comportamento praticamente igual ao pivô no início.
- Conclusão: escolher o último elemento tem o mesmo risco de explosão de comparações em arrays ordenados.

QuickSort com pivô aleatório

- Ordenado: muito eficiente, comparações e movimentos bem menores do que início/fim.
- Aleatório: desempenho muito próximo do pivô no meio ou mediana.
- Observação: boa estratégia para evitar o pior caso ($O(n^2)$) em arrays já ordenados.

QuickSort com pivô mediana

- Ordenado: excelente, comparações e movimentos mínimos.
- Aleatório: desempenho muito próximo do pivô no meio.
- Observação: praticamente garante um bom equilíbrio entre os subarrays, evitando o pior caso.

Resumo geral:

- Para garantir robustez, evitando casos ruins: pivô mediana ou pivô aleatório.
- Para simplicidade, sem se preocupar com arrays já ordenados: pivô do meio é aceitável.
- Evitar pivô início/fim, pois podem gerar $O(n^2)$ facilmente.